



UNIVERSITY OF PISA
MASTER'S DEGREE IN CYBERSECURITY

STREAM CIPHER
COURSE HARDWARE AND EMBEDDED SECURITY

Authors
Federico Pucci
Giovanni Del Bianco

Academic year 2025/2026

Contents

1	Project Specifications	1
1.1	Project Objective	1
1.2	Operating Principles of the Stream Cipher	2
1.3	Analysis of the Technical Specifications	3
1.4	Module Interface	3
2	High-level Model	5
2.1	Purpose of the Golden Model	5
2.2	Encryption Algorithm and Mathematical Formalization	6
2.3	Processing Logic and Keystream Management	6
2.4	User Interface and Test Vector Generation	7
3	RTL Design	10
3.1	RTL Block Diagram and System Architecture	10
3.2	Project Hierarchy	11
3.3	Analysis of the Datapath Submodules	12
3.3.1	The Index Counter (counter)	12
3.3.2	The S-Box Memory (sbox)	12
3.3.3	Encryption Unit (xor_unit)	13
3.4	The Control Unit: FSM Design	13
3.4.1	FSM Architecture	14
3.4.2	FSM Transition Table	14
3.5	Synchronization and Latency Management	15
4	Interface Specifications and Expected Behavior	16
4.1	Description of the Interface and Signal Specification	16
4.1.1	I/O Signal Table	16
4.1.2	Logic Level and Timing Specifications	17
4.2	Operational Protocol (User Manual)	17
4.2.1	Phase 1: Initialization and Reset	17
4.2.2	Phase 2: Session Configuration (Key Loading)	17

4.2.3	Phase 3: Data Processing (Handshake Protocol)	18
4.2.4	Timing Constraints and Critical Cases	18
4.3	Waveform Analysis (ModelSim)	18
4.3.1	Initialization and Synchronous Key Loading	18
4.3.2	Latency Management and Handshake Protocol	19
4.3.3	Sequence Analysis and Counter Progression	19
4.4	Corner Case Analysis	20
4.4.1	User Stall Management (Handshake Stalling)	20
4.4.2	Interruption Through Asynchronous Reset During Operation	20
4.5	Functional Validation Against the Golden Model	21
4.5.1	Test Methodology and Vector Generation	21
4.5.2	Scenario 1 Analysis: Encryption (ASCII to Hex)	21
4.5.3	Scenario 2 Analysis: Decryption (Hex to ASCII)	22
4.5.4	Performance and Throughput Analysis	23
4.5.5	Architectural Efficiency	25
4.5.6	Module Integration Guidelines	26
4.5.7	Validation Conclusions	26
5	Functional Verification	27
5.1	Verification Strategy	27
5.1.1	Integration with the Golden Model	27
5.1.2	Handshake Protocol Verification	28
5.2	Testbench Architecture	28
5.2.1	Interconnection and Local Signals	28
5.2.2	Timing and Reset Signal Generation	29
5.2.3	DUT Instantiation	29
5.2.4	File Variables and Descriptor Management	29
5.3	The File-Based Verification Flow (.tv)	29
5.3.1	Opening and Managing Test Files	30
5.3.2	Simulation Loop and Dynamic Vector Loading	30
5.3.3	Integration with the Verification Task	31
5.3.4	Simulation Session Closure	31
5.4	Test Case Description	31
5.4.1	Basic Test (Sanity Check)	32
5.4.2	Symmetry Test (Invertibility)	32
5.4.3	Corner Cases Testing	33
5.5	Testbench Code Analysis	34
5.5.1	Modularity through Tasks	34
5.5.2	Synchronization and Race Condition Handling	34
5.5.3	Immediate Sampling	34
6	FPGA Implementation Results	35
6.1	Synthesis Environment and FPGA Target	35
6.1.1	Development Software	35
6.1.2	Hardware Target: Intel Cyclone V	35
6.1.3	Design Constraints and Objectives	36
6.1.4	Summary Table of the Environment	36

6.2	Architecture Comparison: LUT vs ROM	36
6.2.1	Prototype A: LUT-Based Implementation (Combinational) . . .	36
6.2.2	Prototype B: ROM-Based Implementation (Synchronous) . . .	37
6.2.3	Motivation for the Comparative Study	37
6.3	Resource Utilization Analysis	37
6.3.1	Optimization of Combinational Logic	38
6.3.2	Register and I/O Pin Analysis	38
6.4	Static Timing Analysis (STA)	38
6.4.1	Maximum Frequency Results (F_{max})	38
6.4.2	Timing Margin Analysis (Slack) and Timing Closure	39
6.4.3	Interpretation of Results	39
6.5	Final Design Justification	40
6.5.1	Resource Utilization Efficiency (Area Efficiency)	40
6.5.2	Scalability and Routing Predictability	41
6.5.3	Throughput vs Frequency Analysis	41
6.5.4	Implementation Conclusions	42

List of Figures

3.1	RTL system architecture: detail of the interconnections between the Control Unit and the Datapath.	10
3.2	FSM state diagram.	13
4.1	Detailed view of the reset phase and loading of the secret key K into the counter.	19
4.2	View of the one clock cycle latency introduced by the <code>ST_ROM_WAIT</code> state.	19
4.3	Complete simulation: evidence of counter progression and stability of the data flow.	20
4.4	Verification of the <code>ST_WAIT</code> state: the system correctly manages the persistence of the <code>in_valid</code> signal.	20
4.5	Asynchronous Reset analysis: immediate interruption of the processing flow during ROM latency.	21

List of Tables

3.1	Internal interconnection signals within the <code>stream_cipher_top</code> module.	11
3.2	Description of the transitions and logical conditions of the <code>ctrl_unit</code> FSM.	15
4.1	Port list of the <code>stream_cipher_top</code> module.	16
6.1	Synthesis environment and hardware target configuration.	36
6.2	Comparison of hardware resources used: LUT vs ROM prototypes . . .	37
6.3	Comparison of Maximum Operating Frequency	38
6.4	Setup and Hold Slack Analysis (ns) with 10.3 ns constraint	39
6.5	Performance Summary and Hardware Resource Comparison (Intel Cyclone V)	41

CHAPTER 1

Project Specifications

1.1 Project Objective

This project aims to design and implement a stream cipher system, known as *Stream Cipher*, optimized for integration on FPGA (*Field Programmable Gate Array*) programmable logic devices. The implementation focuses on the creation of a hardware architecture capable of generating a pseudo-random sequence (*keystream*) that can be combined, via a logical XOR operator, with a stream of *plaintext* to produce the corresponding *ciphertext*.

The core of the design lies in the integration of a *Substitution Box* (S-Box) for non-linear data transformation. Specifically, the primary goal of the project is to demonstrate the effectiveness of a **synchronous ROM based** implementation by rigorously evaluating its space usage and timing performance. This design choice aims to maximize the efficiency in the use of dedicated hardware resources compared to a traditional combinatorial (asynchronous) implementation; in particular, the architecture has been engineered to exploit the *M10K* memory blocks integrated into the Intel FPGA **Cyclone V** target chip (5CGXFC9D6F27C7), thus avoiding excessive consumption of general-purpose programmable logic.

The work documented in the following sections covers the entire digital design flow (*RTL design flow*), structured through the following methodological phases:

- **Behavioral modeling and test vector generation:** Development of a reference model in **Python** aimed at simulating the expected behavior of the cipher and generating the *test vector* files required for subsequent hardware validation.
- **RTL architecture development:** Design and implementation of the code in **SystemVerilog**.

- **Functional verification and logic simulation:** Validation of the design through the **ModelSim** simulation environment. This phase involved the implementation of advanced **testbenches** capable of loading test vectors from external files and comparing the circuit outputs in real time with the expected values to ensure the absence of logical errors.
- **Hardware synthesis and implementation:** Use of the **Intel Quartus Prime** software for synthesis and *Place & Route* targeting the Cyclone V FPGA. The process was automated through a dedicated **Python** script (`build.py`) responsible for managing the compilation flow and the automatic assignment of constraints.
- **Timing analysis and performance optimization:** Execution of *Static Timing Analysis* (STA) using the Quartus **Timing Analyzer** to determine the maximum operating frequency (F_{max}) of the system. Through the iterative modification of constraints in the SDC (*Synopsys Design Constraints*) file and the analysis of *Slack*, **Timing Closure** was successfully achieved even under the most critical thermal and electrical operating scenarios (*Worst-Case*).

1.2 Operating Principles of the Stream Cipher

The operation of the implemented *stream cipher* is based on the generation of a deterministic binary sequence with a pseudo-random appearance, referred to as the **keystream** (K). Unlike block ciphers, which operate on large data segments, a *stream cipher* processes the message one element at a time (in this case, one byte).

The fundamental operation consists of applying the logical *eXclusive OR* (XOR) operator between the plaintext data (P) and the keystream (K):

$$C_i = P_i \oplus K_i \quad (1.1)$$

where C_i represents the resulting ciphertext byte. The symmetric nature of the XOR operation allows the same architecture and the same key to be used for both encryption and decryption, since:

$$P_i = C_i \oplus K_i = (P_i \oplus K_i) \oplus K_i = P_i \oplus (K_i \oplus K_i) = P_i \oplus 0 = P_i \quad (1.2)$$

In the proposed architecture, the keystream generation is entrusted to a structure composed of two main elements:

1. **Counter:** An internal register that increments its value for each incoming data byte. It provides a unique and time varying input.
2. **S-Box (Substitution Box):** A *Look-Up Table* implementing a nonlinear substitution function. Given the counter value as input, the S-Box produces the keystream byte as output. The integrity of the system depends on the complexity of the S-Box internal mapping, whose purpose is to mask the statistical relationships between the counter and the encrypted stream.

This configuration ensures that each byte of the original message is "obfuscated" by a different keystream byte, thereby increasing the robustness of the system against ciphertext analysis attacks.

1.3 Analysis of the Technical Specifications

The system design complies with a set of technical and methodological constraints aimed at ensuring the efficiency and portability of the design on Intel FPGA platforms. The main specifications are summarized in the following points:

- **Hardware Target:** The reference device for synthesis and implementation is the **Intel Cyclone V** FPGA (model 5CGXFC9D6F27C7). This chip provides dedicated memory blocks called *M10K*, which are essential for the efficient implementation of the S-Box.
- **Synchronization and Reset:** The system operates entirely synchronously with respect to the rising edge of the clock signal (*posedge clk*). An active-low asynchronous reset signal (`rst_n`) is provided to return both the system and the internal counter to a safe initial state.
- **Data Handling:** The cipher processes data with an **8-bit** parallelism (byte-oriented architecture). The validity of the input data is managed through a simple *hand-shaking* protocol based on the `data_valid` signal.
- **S-Box Architecture:** The final architecture adopts a **synchronous** approach for S-Box access. This implies the use of internal registers implemented through `always_ff` constructs for signal storage. This choice introduces the need to manage a one-cycle latency for data reading, but drastically optimizes the usage of logic resources.
- **Synthesis Methodology:** For the accurate analysis of the logical *core* only, the compilation flow integrates the automatic assignment of **Virtual Pins**. This technique allows the estimation of the timing performance of the design independently of the intrinsic physical delays of the real hardware package I/O pins.

1.4 Module Interface

The *stream cipher* architecture is encapsulated within a `top-level` module featuring a standard port list for synchronous data processing.

Details on the communication protocol:

- **Reset Management:** On the falling edge of `rst_n`, all internal registers, including the counter and the FSM state, are initialized instantaneously.
- **Input Sampling:** The `in_data` and `data_valid` signals are sampled exclusively on the rising edge of `clk`.
- **Status Signal:** This signal is asserted by the system to notify the external environment that the encryption process of the current byte (or of the entire message, depending on the testbench configuration) has been completed and that the data on `out_data` is stable.
- **Virtual Pins:** During the hardware synthesis phase in Quartus, all signals except the clock were mapped as *Virtual Pins* through dedicated TCL scripts. This configuration made it possible to analyze the occupied logic resources (ALMs) and

the maximum operating frequency (F_{max}) without being affected by the physical limitations of the real FPGA Cyclone V pins.

CHAPTER 2

High-level Model

2.1 Purpose of the Golden Model

In the digital design flow, the hardware verification phase plays a fundamental role in ensuring that the RTL (*Register Transfer Level*) implementation faithfully complies with the logical and mathematical specifications of the system. For this purpose, a software reference model written in Python, called the **Golden Model** (file `GoldenModel.py`), was developed. Its primary task is to serve as a "certainly correct" reference for the behavior of the cipher.

The use of a high-level software model provides several methodological advantages:

- **Automatic Test Vector Generation:** The software processes the input data and generates external files with the `.tv` extension (for example `test1_input.tv` and `test1_expected.tv`), containing respectively the stimuli for the hardware and the corresponding expected results.
- **Objective Verification:** During simulation in **ModelSim**, the hardware testbench is able to read these files and compare, bit by bit, the output produced by the circuit with the value calculated by the Python model, simplifying the identification of possible logical bugs.
- **Test Vector Analysis:** The generated files are stored in the dedicated directory `modelsim/tv/`, ensuring a clean separation between the data generation environment and the hardware simulation environment.

In summary, the *Golden Model* represents the bridge between the algorithmic specification and the hardware implementation, providing an indispensable *Golden Reference* for validating the integrity of the encryption process before the physical FPGA implementation.

2.2 Encryption Algorithm and Mathematical Formalization

The algorithm implemented in the Python model follows the logic of a symmetric stream cipher operating at the single-byte level. The core of the cryptographic transformation lies in the integration of a *Substitution Box* (S-Box) derived from the AES algorithm, used as a *Lookup Table* to generate the *keystream*.

The transformation of each byte of the original message (*plaintext*) into the corresponding encrypted byte (*ciphertext*) is governed by the following mathematical equation:

$$C[i] = P[i] \oplus \text{Sbox}[(K + i) \pmod{256}] \quad (2.1)$$

Where the terms are defined as follows:

- i : positional index of the current byte within the message.
- $P[i]$: value of the i -th byte of the plaintext message.
- K : 8-bit secret key (value in the range 0 to 255) provided by the user.
- Sbox: value extracted from the AES substitution table at the computed index.
- $\oplus$: logical XOR operator (*bitwise eXclusive OR*).
- $C[i]$: value of the resulting i -th byte (ciphertext).

The index used to access the S-Box is computed by adding the secret key K to the positional index i . The application of the modulo 256 operation $\pmod{256}$ ensures that the index always remains within the table addressing range (0–255), effectively implementing a circular addressing mechanism.

A fundamental property of this architecture, directly derived from the use of the XOR operator, is its self-inverse nature. The same script and the same logical function can be used for both encryption and decryption: by applying again the *keystream* generated with the same key K to the ciphertext C , the original plaintext P is mathematically recovered.

2.3 Processing Logic and Keystream Management

The translation of the cryptographic algorithm into software instructions is handled by the core function `process_message(data, key)`. This function takes as input parameters a list of integers representing the message bytes (`data`) and the 8-bit secret key (`key`).

The internal processing follows a sequential flow that models the expected behavior of the hardware:

- **Message Iteration:** The software initializes an empty list `processed_data` and starts a `for` loop that traverses the entire input message array. The loop variable `i` acts as a position counter, functionally replicating the role of the *Counter* in the RTL architecture.

Listing 2.1: Code of the *process_message* function

```
1 def process_message(data, key):
2     processed_data = []
3     for i in range(len(data)):
4         sbbox_index = (key + i) % 256
5         keystream_byte = AES_SBOX[sbbox_index]
6         out_byte = data[i] ^ keystream_byte
7         processed_data.append(out_byte)
8     return processed_data
```

- **Circular Index Management:** For each byte, the S-Box access index is computed using `sbbox_index = (key + i) % 256`. The modulo operator (%) ensures that the index always remains within the range [0, 255], allowing the system to handle messages longer than the S-Box size through circular memory addressing.
- **Keystream Generation:** Once the index is determined, the `keystream_byte` is extracted from the global constant `AES_SBOX`. In hardware terms, this operation corresponds to addressing a synchronous ROM memory.

Listing 2.2: Structure of *AES_SBOX*

```
1 AES_SBOX = [
2     0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0
3     xd7, 0xab, 0x76,
4     ...
5     0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0
6     x54, 0xbb, 0x16
7 ]
```

- **XOR Encryption:** The output byte (`out_byte`) is the result of the bitwise XOR operation (^) between the plaintext byte and the extracted keystream byte. The result is then appended to the list of processed bytes.

This *byte-oriented* software structure ensures that the temporal sequence of the generated test vectors is perfectly aligned with the clock cycles of the final hardware implementation.

2.4 User Interface and Test Vector Generation

The file `GoldenModel.py` has been designed as a versatile interactive tool that guides the user in generating stimuli for hardware simulation. A key feature of the model is the unified management of encryption and decryption operations, made possible by the self-inverse nature of the XOR operator.

- **Input Mode Selection:** At startup, the script requests the selection of the data format, offering two distinct operational paths based on the same core logic:
 1. **Encryption (ASCII Text):** This mode is optimal for testing the system with readable messages (e.g., "Hello World"). Each character of the string is converted into its 8-bit integer equivalent using the `ord()` function, providing a direct input for the cryptographic transformation.

2. **Decryption (Hexadecimal Bytes):** Essential for the hardware debugging phase, this mode allows direct input of hexadecimal bytes separated by spaces. Since the ciphertext often contains values that are not representable as printable ASCII characters, hexadecimal input ensures maximum precision when testing the decryption process.

Listing 2.3: *Execution mode selection log*

```

1
2 =====
3      Stream_Cipher_V1: Golden Model
4 =====
5 This tool generates Test Vector (.tv) files for hardware verification.
6 Note: The same process is used for ENCRYPTION and DECRYPTION.
7
8 Select input format:
9 1) Encryption: ASCII Text (e.g., Hello World)
10 2) Decryption: Hexadecimal Bytes (e.g., 48 65 6c 6c 6f)
11 Selection (1 or 2):

```

- **Execution Duality (Encryption vs Decryption):** From an algorithmic perspective, the script does not distinguish between the two operations. If the user inputs plaintext P with key K , the resulting output is ciphertext C . If the user inputs a previously obtained ciphertext C with the same key K , the XOR property ($C \oplus K = P$) reconstructs the original message. This symmetry allows validation of the entire cryptographic chain using a single set of functions.
- **Key and Environment Configuration:** The user must specify a secret key K (0–255), which is validated to ensure consistency with the 8-bit registers of the FPGA. The script also checks for the existence of the `modelsim/tv/` directory, creating it if necessary using the `os` library, in order to centralize test files.
- **Generation of .tv Files:** The final output consists of two hexadecimal text files in two-digit format (02x), ready to be read by the SystemVerilog `$readmemh` command:
 - `<name>_input.tv`: Contains the stimulus sequence to be applied to the RTL module.
 - `<name>_expected.tv`: Contains the *Golden Reference* computed by the Python model, used by the testbench to validate the hardware output byte by byte.

Execution ends with a console summary showing the comparison between input and output in hexadecimal format and, where possible, a *Text Preview* that replaces non-printable bytes with a special character, enabling immediate visual analysis of the result.

Listing 2.4: *Execution summary log*

```

1
2 -----
3 PROCESSING SUMMARY
4 -----
5 Input (Hex): 48 65 6c 6c 6f 20 57 6f 72 6c 64
6 Output (Hex): 2f 4e 92 bb c4 56 9d ed bb 11 9e
7 Text Preview: /N???V????

```

```
8
9 [SUCCESS] Files generated successfully:
10 -> modelsim/tv/Hello_World_input.tv
11 -> modelsim/tv/Hello_World_expected.tv
12 -----
```

CHAPTER 3

RTL Design

3.1 RTL Block Diagram and System Architecture

The hardware architecture of the *stream cipher* was designed following a modular and hierarchical approach, formalized in the `stream_cipher_top` module. This module acts as a structural container (Figure 3.1), integrating both the data processing units (*Datapath*) and the timing/control management logic (*Control Unit*).

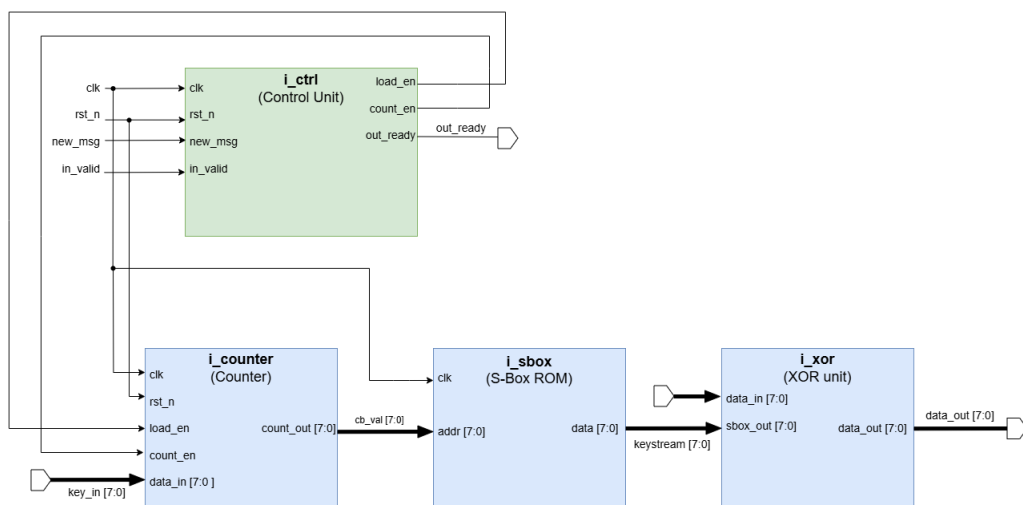


Figure 3.1: RTL system architecture: detail of the interconnections between the Control Unit and the Datapath.

The system organization consists of four main interconnected submodules connected through internal signals (*wires*):

- **i_ctrl (Control Unit):** Represents the "brain" of the system, responsible for coordinating state progression and generating the enable signals for the other modules.
- **i_counter (Counter):** Generates the 8-bit index required for memory addressing, initializing itself with the secret key K .
- **i_sbox (S-Box ROM):** Contains the AES substitution table and is implemented as synchronous memory, requiring a clock signal for address loading.
- **i_xor (XOR Unit):** Represents the final mixing stage between the input data and the *keystream* byte produced by the S-Box.

The interconnection between these blocks enables a smooth and controlled data flow. The `cb_val` signal carries the address generated by the counter toward the ROM, while the `keystream` signal carries the value extracted from the ROM toward the XOR unit. Operational control is guaranteed by the `load_en` and `count_en` lines, which allow the *Control Unit* to manage, respectively, the initialization and increment of the counter according to the current processing state.

This architecture was specifically optimized for synthesis on Intel Cyclone V FPGAs. During synthesis, the counter is translated into registers and adders, the S-Box is inferred as a dedicated **M10K** memory block, the *Control Unit* as a finite state machine, and the XOR unit as a network of elementary logic gates.

3.2 Project Hierarchy

The hardware implementation is structured following a *parent-child* hierarchy, where the `stream_cipher_top` module acts as the top-level entity coordinating four underlying instances. This modularity keeps the code clean, simplifying maintenance and allowing the replacement of individual logic blocks (for example, changing the S-Box table) without rewriting the entire system.

To enable communication between the different units, internal logical signals, referred to as *wires*, have been defined within the *Top-Level*. The following table describes the role of each interconnection signal:

Internal Signal	Width	Interconnection Function
<code>cb_val</code>	8 bit	Connects the output of the <i>Counter</i> to the address input of the <i>S-Box</i> .
<code>keystream</code>	8 bit	Carries the data from the <i>S-Box</i> to the input of the <i>XOR</i> unit.
<code>load_en</code>	1 bit	Control line from the <i>Control Unit</i> to the <i>Counter</i> for key loading.
<code>count_en</code>	1 bit	Control line from the <i>Control Unit</i> to the <i>Counter</i> for increment operations.

Table 3.1: Internal interconnection signals within the `stream_cipher_top` module.

The physical connection between the modules is implemented through explicit *port-mapping* syntax (e.g., `.clk (clk)`), ensuring that all signals are synchronized to the same clock domain.

Separation Between Control and Datapath: The architecture is conceptually divided into two distinct sections operating in synergy:

- **Control Unit (`i_ctrl`):** Manages timing and the *handshaking* protocol with the external environment. It does not process data directly, but decides when data should flow.
- **Datapath (`i_counter`, `i_sbox`, `i_xor`):** Includes the modules that physically manipulate the bits. The flow follows a precise path: the counter generates the position, the ROM retrieves the secret value, and the XOR unit produces the final output.

This clear separation follows modern RTL design paradigms, allowing the synthesizer to independently optimize the combinational logic of the datapath and the FSM state registers.

3.3 Analysis of the Datapath Submodules

This section provides a detailed analysis of the individual logic blocks composing the processing architecture. Each module has been designed to operate in close synergy with the *Control Unit*, ensuring a deterministic and synchronous data flow.

3.3.1 The Index Counter (`counter`)

The `counter` module is an 8-bit register designed to generate the addresses required by the S-Box. Its SystemVerilog implementation provides three main functionalities managed in priority order within an `always_ff` block:

Listing 3.1: Sequential logic of the counter module

```

1 always_ff @(posedge clk or negedge rst_n) begin
2     if (!rst_n) begin
3         count_out <= 8'h00;
4     end else if (load_en) begin
5         count_out <= data_in;
6     end else if (count_en) begin
7         count_out <= count_out + 1'b1;
8     end
9 end

```

- **Asynchronous Reset (`rst_n`):** Has the highest priority. On the falling edge of the signal, the counter value is forced to `8'h00`, returning the system to a known and safe state independently of the clock.
- **Parallel Load (`load_en`):** When asserted by the FSM, the counter ignores its current value and directly loads the secret key K (`data_in`). This operation occurs at the beginning of each new message in order to initialize the encryption sequence.
- **Synchronous Increment (`count_en`):** Allows the index to advance by one unit for each processed byte. Since the register is defined on 8 bits, the modulo 256 operation occurs implicitly through natural *overflow*.

3.3.2 The S-Box Memory (`sbox`)

The substitution unit is implemented as a **synchronous ROM memory**. Unlike a combinational implementation, this choice allows the synthesis software to infer the use of dedicated hardware resources (M10K).

Listing 3.2: *Structure of the synchronous ROM for the AES S-Box*

```
1 always_ff @(posedge clk) begin
2     case (addr)
3         8'h00: data <= 8'h63;
4         8'h01: data <= 8'h7c;
5         // ... lookup logic for the complete AES table ...
6         default: data <= 8'h00;
7     endcase
8 end
```

The code uses a `case` construct inside a sequential clock-driven block. This configuration introduces a fundamental behavior: the address provided by the counter is sampled on the rising edge of the clock, and the corresponding data becomes available at the output only during the following cycle. This **one-cycle latency** is the critical element that guided the design of the finite state machine.

3.3.3 Encryption Unit (`xor_unit`)

The XOR unit represents the final processing stage. It is the only module in the system implemented using purely **combinational** logic (`always_comb`).

Listing 3.3: *Combinational implementation of the XOR operation*

```
1 always_comb begin
2     data_out = data_in ^ sbox_out;
3 end
```

The module receives as input the message byte and the *keystream* byte, instantly returning the result of the bitwise transformation. The choice of combinational logic minimizes the overall system latency. Thanks to the mathematical properties of the XOR operator, this module manages both the encryption and decryption phases.

3.4 The Control Unit: FSM Design

The *Control Unit* represents the decision-making module of the entire system, responsible for coordinating the interaction between external signals and the computational units of the *datapath*. The core of this module is a **Finite State Machine** (Figure 3.2) implemented in SystemVerilog using the two-process style, which guarantees a clear separation between state update logic (sequential) and output/next-state generation logic (combinational).

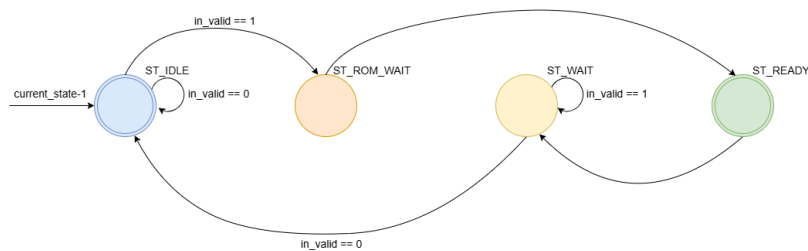


Figure 3.2: *FSM state diagram.*

3.4.1 FSM Architecture

The FSM was defined using a 2-bit enumerated type (`typedef enum`) to map four distinct logical states. The asynchronous reset signal `rst_n` ensures that the system can always be returned to the initial *IDLE* state.

State Analysis:

- **ST_IDLE (Idle State):** In this state, the system waits for external commands. If the `new_msg` signal is detected, the `load_en` line is activated to load the secret key *K* into the counter. When the `in_valid` signal is asserted, indicating the presence of a byte to process, the FSM transitions to the memory read state.
- **ST_ROM_WAIT (Latency Management):** This state is fundamental for interfacing with the synchronous ROM. Since the `sbox` memory requires a clock rising edge to sample the address and an additional cycle to make the output data available, the FSM inserts a forced wait cycle. This guarantees that the XOR unit receives stable data before signaling output validity.
- **ST_READY (Output Validation):** Once the *keystream* is available at the ROM output, the FSM asserts the `out_ready` signal, informing the external environment that the encrypted byte is stable on `data_out`. At the same time, `count_en` is activated to increment the counter and prepare the address for the next byte.
- **ST_WAIT (Handshake):** To prevent the same byte from being processed multiple times if the `in_valid` signal remains high for several clock cycles, a dedicated *handshake* state has been implemented. The system remains blocked in `ST_WAIT` until the user returns `in_valid` to a logic low level, ensuring precise byte by byte synchronization.

The use of an `always_comb` block for the output logic ensures that control signals such as `load_en` and `out_ready` are generated instantaneously according to the current state, minimizing internal propagation delays.

3.4.2 FSM Transition Table

Table 3.2 summarizes the transition conditions between the states of the *Control Unit*, providing the correspondence between the control signals and the logical evolution of the system.

Arrow Path	Condition (Label)	Logical Explanation
Input Arrow (Reset)	$current_state = 1$	Asynchronous initialization of the <i>IDLE</i> state.
ST_IDLE $\rightarrow$ ST_ROM_WAIT	$in_valid = 1$	Detection of a new byte and start of the processing sequence.
ST_IDLE $\rightarrow$ ST_IDLE	$in_valid = 0$	Remain in the waiting state for external stimuli.
ST_ROM_WAIT $\rightarrow$ ST_READY	1 clock cycle	Automatic transition to compensate for ROM latency.
ST_READY $\rightarrow$ ST_WAIT	<i>unconditional</i>	The data is stable and validated; preparation for the handshake phase.
ST_WAIT $\rightarrow$ ST_IDLE	$in_valid = 0$	Handshake completed: the system is ready for a new byte.
ST_WAIT $\rightarrow$ ST_WAIT	$in_valid = 1$	Active protection to prevent multiple processing of the same data.

Table 3.2: Description of the transitions and logical conditions of the *ctrl_unit* FSM.

3.5 Synchronization and Latency Management

The proposed architecture is entirely based on a **synchronous design** paradigm, in which every state transition and every register update are constrained to the rising edge of the clock signal. This choice is fundamental to guarantee signal timing stability (*timing closure*) and to prevent *glitch* phenomena typically associated with complex combinational networks.

The critical aspect of temporal management lies in the interface with the S-Box memory. Since the *sbox* module is implemented to be inferred as an **M10K** memory block (dedicated Block RAM of the Cyclone V FPGA), it operates in registered mode. Consequently, the signal path follows a strict temporal sequence:

1. At cycle T_n , the counter (*i_counter*) presents the *cb_val* address to the ROM input.
2. On the rising edge of the clock leading to cycle T_{n+1} , the ROM samples the address.
3. Only during cycle T_{n+1} does the *keystream* data become stable at the ROM output and can therefore be processed by the XOR unit.

The *Control Unit* manages this behavior through the **ST_ROM_WAIT** state, which acts as a synchronization barrier. Without this mechanism, the system would attempt to validate the output data during the same cycle in which the address is requested, resulting in the generation of an incorrect byte (based on the previous ROM value).

CHAPTER 4

Interface Specifications and Expected Behavior

4.1 Description of the Interface and Signal Specification

This section defines the set of ports of the `stream_cipher_top` module, which constitutes the physical and logical interface of the system. Correct interaction with these signals is fundamental to guarantee deterministic encryption behavior and compliance with the internal synchronization protocols.

4.1.1 I/O Signal Table

The system features an 8-bit data interface and a set of control signals for managing the *handshake* protocol. Table 4.1 summarizes the function of each signal.

Signal	Direction	Width	Description
clk	Input	1	System clock signal (rising edges).
rst_n	Input	1	Active-low asynchronous reset.
new_msg	Input	1	Command used to start a new session (loads the key K).
in_valid	Input	1	Indicates that the input byte on <code>data_in</code> is ready.
key_in	Input	8	8-bit secret key K provided by the user.
data_in	Input	8	Plaintext (or ciphertext) byte to be processed.
out_ready	Output	1	Status signal indicating that the output data is stable.
data_out	Output	8	Resulting byte produced by the XOR operation.

Table 4.1: Port list of the `stream_cipher_top` module.

4.1.2 Logic Level and Timing Specifications

For the correct usage of the module, the operator or master system must comply with the following specifications:

- **Reset (`rst_n`):** Must be held at logic level '0' for at least one clock cycle to guarantee the initialization of all FSM and counter registers.
- **Synchronization:** All input signals (except reset) must be stable before the rising edge of the clock (*setup time*) in order to be correctly sampled by the `i_ctrl` and `i_counter` modules.
- **Data Bus:** The `key_in` and `data_in` signals are interpreted as *unsigned* integers in the range $[0, 255]$. The `data_out` bus presents the result of the combinational XOR operation.

4.2 Operational Protocol (User Manual)

The correct operation of the `Stream_Cipher_V1` cipher requires compliance with a precise operational sequence governed by the *handshake* signals managed by the *Control Unit*. The protocol is divided into three main phases: system initialization, session configuration, and data processing.

4.2.1 Phase 1: Initialization and Reset

Before any operation, the system must be brought into a consistent state through the reset signal:

1. Assert the `rst_n` signal by driving it to logic low.
2. This action instantly forces the FSM into the `ST_IDLE` state and clears the internal counter register.
3. Return `rst_n` to logic '1' to enable normal synchronous operation.

4.2.2 Phase 2: Session Configuration (Key Loading)

To start a new encryption or decryption session, the keystream must be initialized with the secret key K :

1. Set the key value on the `key_in[7:0]` bus.
2. Assert the `new_msg` signal for at least one clock cycle.
3. The *Control Unit* detects the command and internally activates `load_en`, forcing the counter to load the value present on `key_in`.
4. Once the key has been loaded, the system is ready to process the first byte of the message.

4.2.3 Phase 3: Data Processing (Handshake Protocol)

The transfer of each individual byte follows an *interlocked* synchronization protocol to guarantee that no data is skipped or duplicated:

1. **Data Transmission:** The user places the byte on the `data_in[7:0]` bus and asserts the `in_valid` signal.
2. **Internal Processing:** The FSM transitions into the `ST_ROM_WAIT` state to wait for the output data from the synchronous ROM to become stable.
3. **Output Sampling:** When the `out_ready` signal is driven to logic '1' by the module, the result of the operation is stable on the `data_out[7:0]` bus. The user may sample the data at this moment.
4. **Handshake Completion:** The user **must** drive `in_valid` back to logic '0'. The system will remain stalled in the `ST_WAIT` state until `in_valid` returns low, guaranteeing that each byte is processed exactly once.
5. **Subsequent Bytes:** For the following bytes, repeat the steps starting from point 1. The counter will automatically increment by one unit for each completed cycle.

4.2.4 Timing Constraints and Critical Cases

To guarantee data integrity, the operator must comply with the following constraints:

- **Persistence of `in_valid`:** The `in_valid` signal must remain high until `out_ready` is asserted, in order to avoid incomplete state transitions.
- **Mandatory Release:** Failing to release `in_valid` after the `out_ready` signal will prevent the FSM from returning to `ST_IDLE`, effectively blocking the system for subsequent bytes.
- **Key Change:** If it is necessary to change the key K during a session, the `new_msg` signal must be asserted again. This operation overwrites the current counter index with the new value.

4.3 Waveform Analysis (ModelSim)

This section analyzes the dynamic behavior of the system through the timing diagrams obtained in the ModelSim environment. The simulation confirms the perfect synchronization between the control unit and the datapath, validating the architectural choices discussed in Chapter 3.

4.3.1 Initialization and Synchronous Key Loading

Figure 4.1 illustrates the system startup phase. Initially, the asynchronous reset signal `rst_n` is held at logic low, forcing the FSM into the `ST_IDLE` state and clearing the index counter (`count_out = 00h`).

After the reset is released, the system remains waiting for external stimuli. The assertion of the `new_msg` signal for a single clock cycle activates the parallel loading logic: on the following rising edge, the value present on the `key_in` bus (in this case

10) is sampled and transferred into the counter register. From this moment, the system is initialized and ready for the first encryption operation starting from the specified index K .

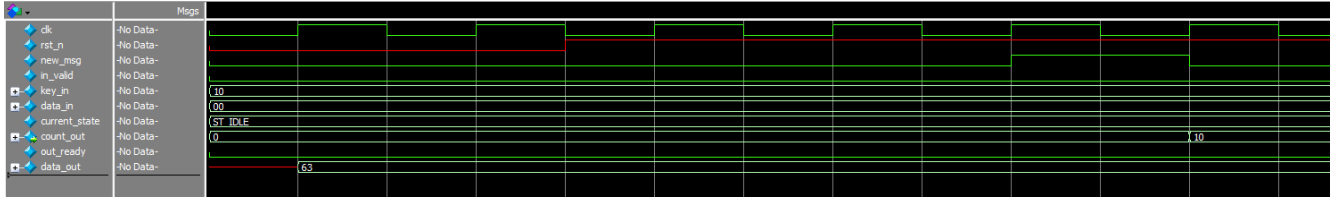


Figure 4.1: Detailed view of the reset phase and loading of the secret key K into the counter.

4.3.2 Latency Management and Handshake Protocol

The most critical behavior of the design lies in the management of the synchronous ROM memory, analyzed in detail in Figure 4.2. When an input data arrives ($in_valid = 1$), the FSM immediately transitions into the `ST_ROM_WAIT` state.

As can be observed, during this cycle the address is stably presented to the S-Box, but the `out_ready` signal still remains at logic '0'. This "temporal barrier" is necessary because the memory requires one clock edge to sample the address and an additional propagation delay to make the output data available. Only during the following cycle, with the transition to the `ST_READY` state, is the `out_ready` signal asserted, confirming that the value on `data_out` (resulting from the XOR operation between `data_in` and the newly extracted *keystream*) is valid and can be sampled by the system.

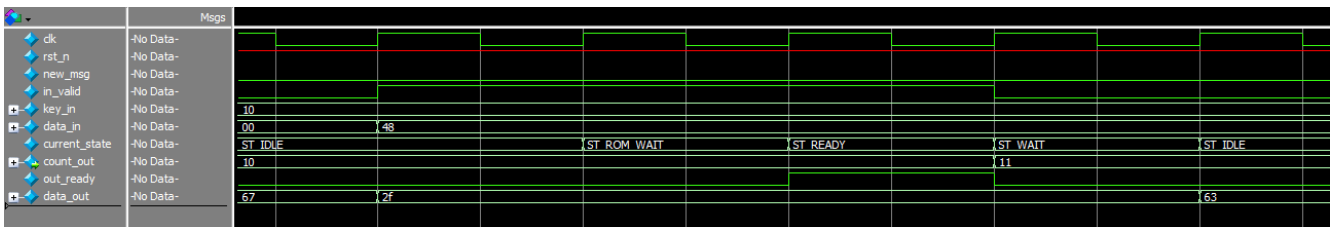


Figure 4.2: View of the one clock cycle latency introduced by the `ST_ROM_WAIT` state.

4.3.3 Sequence Analysis and Counter Progression

Figure 4.3 provides an overall view of the processing of a multi-byte message. This waveform allows the deterministic progression of the index counter to be validated.

It can be clearly observed that, after each completed handshake cycle (return to `ST_IDLE`), the `count_out` address increments linearly (10, 11, 12, 13...). This guarantees that each byte of the original message is processed using a different *keystream* byte, preserving the "stream cipher" nature of the algorithm. The stability of the data buses and the cleanliness of the control signals confirm the absence of *glitch* phenomena or critical races during the transition between consecutive bytes, ensuring a constant *throughput* once the handshake has been completed.

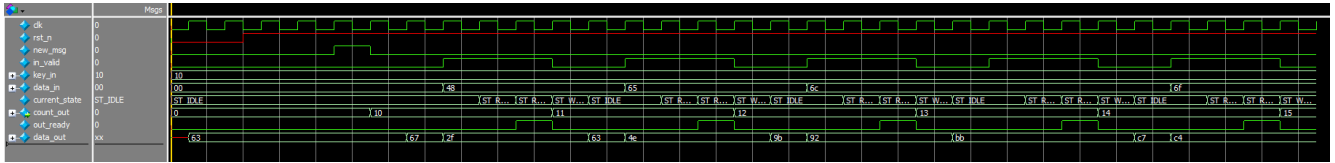


Figure 4.3: Complete simulation: evidence of counter progression and stability of the data flow.

4.4 Corner Case Analysis

In addition to validating the standard operational flow, the project was subjected to stress testing in order to verify its resilience under critical scenarios. The objective is to guarantee that the control logic never enters undefined states and that the communication protocol is "error-proof".

4.4.1 User Stall Management (Handshake Stalling)

In a real system, the master entity (software or other hardware) may not be able to sample the data immediately or may keep the validity signal asserted for multiple clock cycles. Figure 4.4 analyzes this scenario, referred to as "Slow User".

As highlighted in the waveform, once the data is ready and the `out_ready` signal is driven to logic '1', the FSM transitions into the `ST_WAIT` state. During this phase, even if the clock continues oscillating, the system remains "frozen": the index counter (`count_out`) maintains its current value and no new data is processed. This synchronization barrier is fundamental to prevent the "double counting" phenomenon, guaranteeing that each byte is encrypted exactly once. The return to the `ST_IDLE` state occurs only when the user de-asserts `in_valid`, correctly resetting the handshake protocol for the next byte.

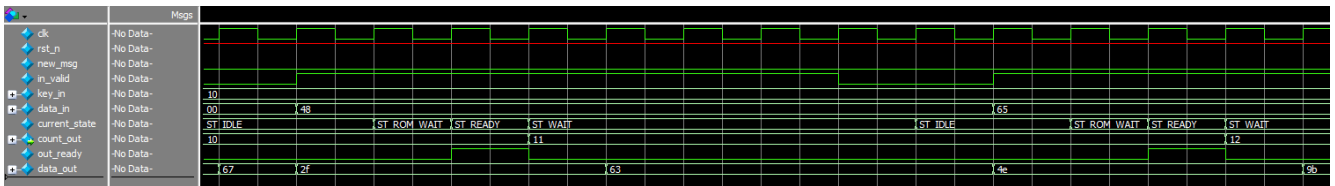


Figure 4.4: Verification of the `ST_WAIT` state: the system correctly manages the persistence of the `in_valid` signal.

4.4.2 Interruption Through Asynchronous Reset During Operation

One of the strictest safety requirements for a hardware module is the ability to instantly return to a known state in case of emergency. Figure 4.5 shows the behavior of the system when an asynchronous reset is applied during computation (specifically during the `ST_ROM_WAIT` state).

The timing analysis reveals that the transition to `ST_IDLE` and the clearing of the `count_out` bus occur exactly on the falling edge of the `rst_n` signal, without waiting for the next rising edge of the clock. This "asynchronous" behavior is crucial: it guarantees that, regardless of the operating frequency or the current FSM state, the

system can be reset within a fraction of a clock cycle, immediately interrupting any ongoing encryption operation and protecting the internal registers.

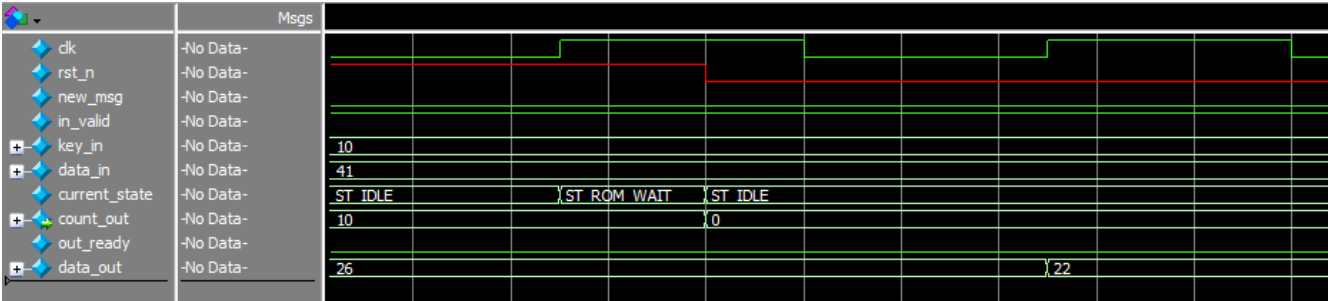


Figure 4.5: Asynchronous Reset analysis: immediate interruption of the processing flow during ROM latency.

4.5 Functional Validation Against the Golden Model

The final objective of the testing phase is the *end-to-end* validation process, aimed at guaranteeing that the SystemVerilog hardware implementation is semantically equivalent to the mathematical model (*Golden Model*) developed in Python. This section serves as a verification manual, illustrating how the data is processed, the module response timing, and the proof of perfect symmetry between encryption and decryption operations.

4.5.1 Test Methodology and Vector Generation

The verification flow is based on the use of *Test Vectors* (.tv files) to automate the comparison between the software model and the hardware implementation. The procedure follows three rigorous steps:

1. **Software Generation:** The Python script computes the expected values and generates two files: `input.tv` (hardware stimuli) and `expected.tv` (expected results).
2. **Data Injection:** The Testbench reads the files and applies the signals to the module interface while respecting the *handshake* protocol through the assertion of the `in_valid` signal.
3. **Timing Monitoring:** The Testbench samples `data_out` only when the `out_ready` signal is high, measuring the elapsed time (t) for each transaction.

4.5.2 Scenario 1 Analysis: Encryption (ASCII to Hex)

In this scenario, the operator provides the test string “Hello” and a secret key $K = 10$. The *Golden Model* defines the expected behavior that the hardware must replicate.

Listing 4.1: Python Golden Model Log: Encryption Phase.

```

1
2 Stream_Cipher_V1: Golden Model
3
4 =====

```

```

5 Select input format: 1) Encryption: ASCII Text
6 Enter the string: Hello
7 Enter the 8-bit Secret Key K: 10
8
9 PROCESSING SUMMARY
10
11 Input (Hex): 48 65 6c 6c 6f
12 Output (Hex): 2f 4e 92 bb c4
13
14 [SUCCESS] Files generated:
15 Hello_input.tv
16 Hello_expected.tv

```

The ModelSim simulator executes the generated vectors. The console log below includes the timestamp (t) expressed in picoseconds (ps).

Listing 4.2: *ModelSim Log: Timing and Functional Validation (Encryption).*

```

1
2 # --- Unified Simulation Started with .tv Files ---
3
4 # [OK] t= 95000 | In:48 -> Out:2f
5 # [OK] t= 145000 | In:65 -> Out:4e
6 # [OK] t= 195000 | In:6c -> Out:92
7 # [OK] t= 245000 | In:6c -> Out:bb
8 # [OK] t= 295000 | In:6f -> Out:c4
9
10 # --- Test Completed Successfully ---

```

4.5.3 Scenario 2 Analysis: Decryption (Hex to ASCII)

To demonstrate the symmetric nature of the system, the encrypted bytes previously obtained (2f 4e 92 bb c4) were used as input for the decryption operation using the same key.

Listing 4.3: *Python Golden Model Log: Decryption Phase.*

```

1
2 Stream_Cipher_V1: Golden Model
3
4 =====
5 Select input format: 2) Decryption: Hexadecimal Bytes
6 Enter hex bytes: 2f 4e 92 bb c4
7 Enter the 8-bit Secret Key K: 10
8
9 PROCESSING SUMMARY
10
11 Input (Hex): 2f 4e 92 bb c4
12 Output (Hex): 48 65 6c 6c 6f
13
14 Text Preview: Hello

```

The ModelSim log confirms that the hardware is capable of perfectly reversing the process, recovering the original ASCII codes of the word “Hello”.

Listing 4.4: *ModelSim Log: Timing and Functional Validation (Decryption).*

```

1
2 # --- Unified Simulation Started with .tv Files ---
3
4 # [OK] t= 95000 | In:2f -> Out:48
5 # [OK] t= 145000 | In:4e -> Out:65
6 # [OK] t= 195000 | In:92 -> Out:6c
7 # [OK] t= 245000 | In:bb -> Out:6c
8 # [OK] t= 295000 | In:c4 -> Out:6f

```

4.5.4 Performance and Throughput Analysis

Based on the data extracted from the simulation performed in ModelSim, it is possible to accurately define the performance profile of the hardware module by analyzing both its overall latency and operational throughput.

Total Processing Latency

For a message consisting of 5 bytes (the string “Hello”), the total measured simulation time from the initial reset to the completion of the final operation is approximately 315 ns.

This time interval includes:

- the initial asynchronous reset phase;
- system stabilization after reset;
- loading of the secret key;
- serial processing of individual bytes;
- final simulation completion.

The detailed timing breakdown is as follows:

Time Interval	Operation
0 – 20 ns	Asynchronous reset active
20 – 40 ns	Post-reset stabilization
40 – 60 ns	Key loading (<i>new_msg</i>)
60 – 310 ns	Processing of the 5 bytes
310 – 315 ns	Simulation termination

The effective payload processing therefore requires:

$$5 \text{ bytes} \times 50 \text{ ns/byte} = 250 \text{ ns}$$

confirming the perfectly regular timing behavior of the datapath.

Scalability Considerations

The architecture exhibits linear scalability with respect to message length, since each byte is processed independently through a deterministic pipeline. The total execution time T_{total} can be expressed as:

$$T_{total} = T_{init} + N \cdot T_{byte}$$

where:

- T_{init} is the initialization overhead (reset + key loading),
- N is the number of processed bytes,

- T_{byte} is the per-byte processing time (approximately constant).

This formulation confirms that the system does not introduce additional algorithmic complexity with increasing input size, making it suitable for streaming applications and real-time data encryption.

Latency and Throughput analysis

Considering a system clock frequency of $f_{CLK} = 100 \text{ MHz}$, the clock period can be computed using the general relation:

$$T_{CLK} = \frac{1}{f_{CLK}} \quad (4.1)$$

Substituting the values of the designed system:

$$T_{CLK} = \frac{1}{100 \text{ MHz}} = 10 \text{ ns}$$

The processing latency of a single byte can be expressed as:

$$L = L_{CC} \cdot T_{CLK} \quad (4.2)$$

where:

- L represents the total latency;
- L_{CC} represents the number of clock cycles required;
- T_{CLK} represents the clock period.

Since:

$$T_{CLK} = \frac{1}{f_{CLK}} \quad (4.3)$$

the equivalent formulation is obtained:

$$L = \frac{L_{CC}}{f_{CLK}} \quad (4.4)$$

In the designed system, each transaction requires 5 clock cycles; therefore:

$$L = \frac{5}{100 \times 10^6} = 50 \text{ ns}$$

each transaction therefore requires:

$$\frac{50 \text{ ns}}{10 \text{ ns}} = 5 \text{ clock cycles}$$

The pipeline behavior of the finite state machine can be described as follows:

1. **Cycle 1:** Input acquisition through assertion of `in_valid`.
2. **Cycle 2:** State `ST_ROM_WAIT`, required to handle the ROM access latency.

3. **Cycle 3:** State `ST_READY`; the `out_ready` signal is asserted and the encrypted/decrypted byte becomes available.
4. **Cycle 4:** State `ST_WAIT`, dedicated to handshake synchronization.
5. **Cycle 5:** Return to `ST_IDLE`, ready for the next processing operation.

The theoretical throughput of the system can be expressed as:

$$\text{Throughput} = \frac{\text{Amount of processed data}}{\text{Processing time}} \quad (4.5)$$

In the specific case of this project, the module processes 1 byte every 50 ns; therefore:

$$\text{Throughput} = \frac{1 \text{ Byte}}{50 \times 10^{-9} \text{ s}} = 20 \times 10^6 \text{ Byte/s}$$

i.e.:

$$\boxed{20 \text{ MB/s}}$$

Equivalently, in terms of bit-rate:

$$20 \text{ MB/s} \times 8 = 160 \text{ Mbit/s}$$

The main performance indicators of the system can be summarized as follows:

- **Latency per byte:** $\approx 50 \text{ ns}$
- **Total latency:** $\approx 315 \text{ ns}$
- **Throughput:** $\approx 20 \text{ MB/s}$
- **Architecture type:** Fully synchronous streaming datapath

The results confirm that the design achieves a balanced trade-off between simplicity, determinism, and processing speed, making it suitable for FPGA-based lightweight cryptographic applications.

4.5.5 Architectural Efficiency

Despite the latency introduced by the ROM memory, which enforces a mandatory wait cycle, the architecture maintains a constant and deterministic operating rate.

In particular, the finite state machine guarantees:

- correct synchronization between datapath and control unit;
- absence of timing collisions;
- constant throughput independent of input data;
- fully deterministic behavior.

These properties make the module suitable for real-time stream-oriented encryption applications.

4.5.6 Module Integration Guidelines

To maintain the maximum throughput of 20 MB/s, the external system must strictly follow the handshake protocol.

In particular:

- the `in_valid` signal must be held high until `out_ready` is asserted;
- the value on `data_out` must be sampled only when `out_ready` is high;
- a new byte can be submitted every 5 clock cycles.

Any delay in re-asserting `in_valid` or incorrect handshake handling will result in a proportional reduction of the effective system throughput.

4.5.7 Validation Conclusions

The complete bit by bit match between the Python model logs and the ModelSim simulation results, combined with the verification of the corner cases discussed in previous sections (stall and reset scenarios), confirms that the RTL design is robust, deterministic, and fully compliant with the reference algorithm.

The system also demonstrates correct handling of ROM latency and synchronization protocols, making it suitable for future implementation on Intel Cyclone V FPGA devices.

CHAPTER 5

Functional Verification

5.1 Verification Strategy

The functional verification of the `stream_cipher_top` module follows a *Black-Box Testing* methodology. The system is stimulated exclusively through its interface ports, treating the internal logic as a closed entity whose correctness is assessed only in terms of output conformity with respect to the applied inputs.

5.1.1 Integration with the Golden Model

The verification strategy relies on an external Python-based *Golden Model*. The operational flow is based on vector files (`.tv`) containing input stimuli and their corresponding expected outputs.

This approach ensures strict independence between the software reference model and the RTL implementation: any logical error in the hardware description is immediately detected by the testbench, since the expected behavior is generated in a separate computational environment.

The validation process is structured into the following phases:

1. generation of test vectors using the Python model;
2. automatic loading of `input.tv` and `expected.tv` files;
3. application of stimuli to the hardware module;
4. automatic comparison between actual and expected outputs;
5. real-time generation of diagnostic reports.

The adoption of this methodology enables a fully automated, reproducible, and scalable verification flow.

5.1.2 Handshake Protocol Verification

The verification process does not only assess the correctness of the processed data, but also rigorously validates the communication protocol based on the handshake signals `in_valid` and `out_ready`.

The testbench employs temporal synchronization constructs such as:

- `wait()`;
- `@(posedge clk)`;
- conditional synchronization based on module states.

These constructs allow a realistic simulation of a master device interacting with the hardware cipher module.

In particular:

- input data is considered valid only when `in_valid` is asserted;
- output data is sampled exclusively when `out_ready` is asserted high;
- the testbench verifies correct timing compliance throughout all transactions.

This strategy enables validation not only of the algorithmic correctness of the cipher, but also of the temporal robustness of the hardware interface and the correct synchronization between datapath and control unit.

5.2 Testbench Architecture

The simulation environment has been structured to wrap the *Design Under Test* (DUT) within a framework capable of autonomously handling timing, data injection, and response monitoring. The testbench architecture is modular and is based on a main instance (`tb_stream_cipher.sv`) supported by specialized versions for corner-case analysis.

5.2.1 Interconnection and Local Signals

To interface the testbench with the `stream_cipher_top` module, local signals of type `logic` have been defined to replicate the DUT port structure. These signals act as “wires” that carry test vectors to the inputs and capture results from the outputs.

Listing 5.1: *Definition of interconnection signals in the testbench.*

```
1 // --- Local Signals (Interconnects to the DUT) ---
2 logic      clk;
3 logic      rst_n;
4 logic      new_msg;
5 logic      in_valid;
6 logic [7:0] key_in;
7 logic [7:0] data_in;
8 logic      out_ready;
9 logic [7:0] data_out;
```

5.2.2 Timing and Reset Signal Generation

The temporal core of the testbench is the clock generator, configured to operate at 100 MHz (10 ns period). In SystemVerilog, this is achieved using an `always` construct that toggles the clock signal every 5 ns.

Listing 5.2: 100MHz clock generator.

```
1 // Clock Generator: 5ns high, 5ns low -> 10ns period = 100MHz
2 always #5 clk = ~clk;
```

Reset management is handled within the `initial` block, where the `rst_n` signal is initially asserted to '0' to clear the system state and then released after a 20 ns interval, ensuring a stable startup condition.

Listing 5.3: Reset management

```
1     #20 rst_n = 1;
2     repeat(2) @(posedge clk);
```

5.2.3 DUT Instantiation

The `stream_cipher_top` module is instantiated inside the testbench by mapping the DUT ports to the previously defined local signals. This configuration allows the testbench to have full control over input data (`data_in`, `key_in`) and synchronization signals (`new_msg`, `in_valid`).

Listing 5.4: Top-level module instantiation inside the testbench.

```
1 stream_cipher_top dut (
2     .clk      (clk),
3     .rst_n    (rst_n),
4     .new_msg   (new_msg),
5     .in_valid  (in_valid),
6     .key_in    (key_in),
7     .data_in   (data_in),
8     .out_ready (out_ready),
9     .data_out  (data_out)
10 );
```

5.2.4 File Variables and Descriptor Management

To enable communication between the Python Golden Model and ModelSim, the testbench declares dedicated variables for *file I/O* management. File descriptors are used to open `.tv` files in read mode, while auxiliary variables temporarily store values read during the simulation loop.

Listing 5.5: Variables for test vector file handling.

```
1 // --- File Handling Variables ---
2 int fd_in, fd_exp;           // File descriptors
3 logic [7:0] val_in, val_exp; // Storage for read values
4 int status;                 // Return status of fscanf
```

5.3 The File-Based Verification Flow (.tv)

The core of the verification strategy lies in the ability of the testbench to correctly interpret the test vectors generated by the *Golden Model*. This approach allows a complete

separation between stimulus generation and hardware simulation logic, enabling the testing of complex messages, arbitrary strings, and random byte sequences.

The use of vector files (.tv) also makes the verification process highly modular, automatable, and easily extendable to large scale test campaigns.

5.3.1 Opening and Managing Test Files

Inside the `initial` block, the testbench opens the files:

- `input.tv`, containing the input data;
- `expected.tv`, containing the expected results generated by the Python model.

File opening is performed using the *system task* `$fopen`. A control mechanism has also been implemented to verify the presence of the files in the simulation directory (`modelsim/tv/`), preventing simulation startup when required vectors are missing.

Listing 5.6: File opening and error handling in the initial block.

```
1 // --- Open Test Vector Files ---
2
3 fd_in  = $fopen("tv/Hello_input.tv", "r");
4 fd_exp = $fopen("tv/Hello_expected.tv", "r");
5
6 if (fd_in == 0 || fd_exp == 0) begin
7     $display("ERROR: tv_files_not_found_in_modelsim/tv/_folder!");
8     $finish;
9 end
```

If one of the file descriptors is null, the testbench immediately terminates the simulation using the `$finish` system task, avoiding execution on missing or invalid data.

5.3.2 Simulation Loop and Dynamic Vector Loading

After the reset phase and key loading, the testbench enters a `while` loop that remains active until the end of the input file is reached, detected using the `$feof` system task.

For each iteration, the following operations are performed:

1. reading a hexadecimal byte from `input.tv`;
2. reading the corresponding expected value from `expected.tv`;
3. checking the read status to exclude parsing errors;
4. invoking the automatic verification task.

Data reading is performed using the `$fscanf` system task, which automatically converts the hexadecimal value from the file into the format required by the simulation.

Listing 5.7: Main loop for test vector loading and verification.

```
1 // --- Main Simulation Loop ---
2
3 while (!$feof(fd_in)) begin
4
5     // Read hexadecimal values from the files
6     status = $fscanf(fd_in, "%h\n", val_in);
7     status = $fscanf(fd_exp, "%h\n", val_exp);
8
```

```

9      // Verify that the read operation completed correctly
10     if (status != -1) begin
11
12         // Send the byte and verify the response
13         send_and_check(val_in , val_exp);
14
15     end
16
17 end

```

This approach allows the testbench to operate dynamically on files of arbitrary length, without requiring structural modifications to the verification code.

5.3.3 Integration with the Verification Task

The `send_and_check()` procedure represents the core execution unit of the automated verification flow. This task implements the handshake protocol required to send data to the hardware module and subsequently verifies the correctness of the produced output.

The task is responsible for:

- applying the input byte;
- asserting the `in_valid` signal;
- waiting for the assertion of `out_ready`;
- sampling `data_out`;
- comparing the obtained result with the expected value.

The entire procedure is fully automated and synchronized with the system clock.

5.3.4 Simulation Session Closure

At the end of the processing of all test vectors, the testbench closes the file descriptors using the `$fclose` system task.

This step ensures:

- proper release of system resources;
- complete saving of simulation logs;
- clean termination of the verification session.

Explicit file closure is a fundamental best practice in the design of reliable and scalable simulation environments.

5.4 Test Case Description

The verification plan has been organized into different categories of tests, designed to validate both the nominal behavior of the cipher and its robustness under abnormal or degraded operating conditions.

The goal of the simulation campaign is not limited to algorithmic correctness alone, but also includes:

- verification of timing synchronization;
- validation of the handshake protocol;
- validation of the finite state machine;
- handling of corner cases;
- resilience to asynchronous resets.

5.4.1 Basic Test (Sanity Check)

The *Sanity Check* represents the first level of functional validation of the system and aims to verify the correct operation of the standard encryption flow.

The test procedure includes the following steps:

1. application of the asynchronous reset;
2. initialization of the state machine;
3. loading of the secret key K through the `new_msg` signal;
4. serial transmission of bytes through the handshake protocol;
5. automatic verification of the produced output.

For each transaction, the testbench automatically compares the value returned by the hardware with the expected value generated by the *Golden Model*. The verification result is immediately displayed on the simulation console.

Listing 5.8: *Reporting logic implemented in the main testbench.*

```

1  if (data_out != bexp) begin
2
3      $display (" [FAILED]_t=%t_l_In:%h_l_Out:%h_l_Expected:%h" ,
4                $time , bin , data_out , bexp);
5
6  end
7  else begin
8
9      $display (" [OK]_t=%t_l_In:%h->_l_Out:%h" ,
10               $time , bin , data_out);
11
12 end

```

The use of the `!=` operator allows the detection of undefined (X) or high-impedance (Z) values as well, increasing the reliability of the verification process.

5.4.2 Symmetry Test (Invertibility)

Since the project is based on a *Stream Cipher* algorithm, the same operation must be applicable for both encryption and decryption.

The invertibility property has been validated using test vectors generated by the *Golden Model* in decryption mode. In particular:

- the encrypted text is provided as input to the hardware module;
- the same secret key K is reloaded into the system;

- the testbench compares the produced output with the original plaintext.

The test is considered successful if the recovered text matches the original message bit by bit.

This test confirms that:

- the generated keystream is deterministic;
- the pseudo random sequence is fully reproducible;
- the datapath correctly implements the symmetric property of the XOR operation.

5.4.3 Corner Cases Testing

To evaluate the robustness of the control logic and the FSM, dedicated testbenches have been developed for critical operating conditions.

Asynchronous Reset During Operation

The `tb_stream_cipher_Reset.sv` testbench has been designed to verify the correct handling of an asynchronous reset triggered during normal module operation.

The main objective is to demonstrate that:

- all internal registers are immediately reinitialized;
- the FSM returns instantly to the `ST_IDLE` state;
- the system does not enter metastable or inconsistent states.

To increase test severity, the reset is triggered at an arbitrary time, not synchronized with the clock rising edge.

Listing 5.9: *Injection of an asynchronous reset out of phase with the clock.*

```

1 // Wait 3 ns after the clock edge
2 #3;
3
4 $display("!!!_TRIGGERING_ASYNCHRONOUS_RESET_NOW_!!!");
5
6 rst_n = 0;
```

This scenario represents a particularly critical condition in real digital systems, where external events may interrupt circuit operation at any time.

Slow User Scenario (Handshake Stalling)

The `tb_stream_cipher_Slow.sv` testbench simulates a user that intentionally delays the deassertion of the `in_valid` signal.

This scenario verifies the correctness of the `ST_WAIT` state of the FSM and the robustness of the handshake protocol.

In particular, the test demonstrates that:

- the system does not process the same byte multiple times;
- the internal counter does not advance prematurely;
- the datapath remains stable during the stall condition;

- synchronization is correctly maintained.

The stall condition is simulated by keeping the `in_valid` signal asserted for multiple consecutive clock cycles.

Listing 5.10: *Simulation of a forced handshake stall.*

```

1 // Keep in_valid asserted for 5 additional clock cycles
2 repeat(5) @(posedge clk);
3
4 in_valid = 0;
```

Passing this test confirms that the implemented communication protocol is robust even in the presence of slow or temporarily unresponsive external peripherals.

5.5 Testbench Code Analysis

The quality of verification depends not only on the test vectors, but also on the precision with which the testbench code interacts with the *Design Under Test* (DUT). In this section, the technical solutions adopted to handle timing and modularity are analyzed.

5.5.1 Modularity through Tasks

To avoid code redundancy and facilitate maintainability, the stimulation and control logic has been encapsulated into a `task` named `send_and_check`. This design choice allows the handshake protocol to be abstracted, making the main simulation loop more readable and focused on the data flow.

5.5.2 Synchronization and Race Condition Handling

One of the main challenges in hardware simulation is the management of *race conditions*. To ensure correct data injection and sampling, the testbench uses the `@(posedge clk)` operator to align every action with the rising edge of the system clock.

Listing 5.11: *Synchronization logic inside the verification task.*

```

1 task send_and_check(input logic [7:0] bin ,
2                     input logic [7:0] bexp);
3
4     @(posedge clk);
5     data_in = bin;
6     in_valid = 1;
7
8     wait (out_ready == 1);
9
10 endtask
```

The statement `wait (out_ready == 1)` is fundamental: it allows the testbench to adaptively handle the latency introduced by the `ST_ROM_WAIT` state of the FSM, without relying on hard-coded delays that would make the testbench fragile and non-portable.

5.5.3 Immediate Sampling

Correctness checking is performed through immediate sampling while the `out_ready` signal is still asserted. This occurs before the next clock edge advances the state machine or increments the counter, ensuring that the verified value corresponds exactly to the current input transaction.

CHAPTER 6

FPGA Implementation Results

6.1 Synthesis Environment and FPGA Target

The transition from the behavioral SystemVerilog description to the physical circuit implementation was carried out using a professional design flow based on the **Intel FPGA** ecosystem. This phase, known as logic synthesis and *Place-and-Route*, is essential to determine whether the temporal and logical abstractions defined in the design are physically realizable on silicon.

6.1.1 Development Software

The entire implementation process was carried out using **Intel Quartus Prime Lite Edition** (Version 25.1std.0, Build 1129). Quartus Prime integrates all the necessary engines for technology mapping, logic network optimization, and delay analysis. Within this suite, two main tools were used:

- **Analysis & Synthesis:** for translating RTL into an optimized logic gate network (*netlist*) tailored to the specific FPGA architecture.
- **Timing Analyzer:** for *Static Timing Analysis* (STA), used to verify that all critical paths satisfy the timing constraints imposed by the system clock.

6.1.2 Hardware Target: Intel Cyclone V

The selected device for implementation is an FPGA from the **Cyclone V GX** family, specifically the **5CGXFC9D6F27C7** model. This family was chosen due to its excellent balance between low power consumption and architectural flexibility. The key characteristics of this target that influence the design are:

- **ALMs (Adaptive Logic Modules):** the basic configurable logic units, highly efficient for mapping complex combinational functions.
- **M10K memory blocks:** integrated 10 Kbit memory blocks that allow the S-Box (2048 bits total) to be implemented without consuming general purpose logic resources.
- **Deterministic routing:** essential to ensure that cipher performance remains consistent across different compilations.

6.1.3 Design Constraints and Objectives

During compilation, a strict clock constraint was imposed using a timing specification file (*SDC – Synopsys Design Constraints*). The primary target was set to an operating frequency of **100 MHz**, corresponding to a clock period:

$$T_{clk} = 10 \text{ ns}$$

Optimization was configured under a **Balanced Performance** profile, aiming to achieve the best trade-off between area utilization (number of ALMs) and maximum operating frequency (F_{max}). This approach ensures that the design is not only fast but also scalable for potential integration of multiple cipher instances on the same chip (parallelism).

6.1.4 Summary Table of the Environment

Table 6.1: *Synthesis environment and hardware target configuration.*

Component	Specifications
Synthesis Software	Intel Quartus Prime Lite Edition v25.1
FPGA Family	Cyclone V GX
Device Part Number	5CGXFC9D6F27C7
Timing Analyzer	TimeQuest (STA)
Clock Constraint	100 MHz (10 ns)
Speed Grade	7 (Standard Grade)

6.2 Architecture Comparison: LUT vs ROM

To identify the most efficient implementation of the cipher on the FPGA target, the synthesis process was applied to two different variants of the `stream_cipher_top` module. Both versions are functionally identical and have been validated using the same testbench, but they differ significantly in the technological mapping of the S-Box.

6.2.1 Prototype A: LUT-Based Implementation (Combinational)

The first prototype, named `Stream_cipher_v1_LUT`, treats the substitution table as a purely combinational function. In this configuration, the S-Box is described using `case` constructs or direct assignments that the synthesis tool translates into a network of **LUT (Look-Up Tables)** and distributed multiplexers.

- **Advantage:** the absence of a memory synchronization clock allows the keystream value to be obtained immediately after address computation.
- **Disadvantage:** the extensive use of generic logic to implement the 256-byte S-Box leads to higher area consumption and potential routing complexity.

6.2.2 Prototype B: ROM-Based Implementation (Synchronous)

The second prototype, `Stream_cipher_v1_ROM`, represents the final architecture discussed in previous chapters. Here, the S-Box is implemented as a synchronous ROM memory, exploiting the FPGA's embedded memory blocks (M10K). This choice requires the introduction of a wait state (`ST_ROM_WAIT`) in the FSM, but it results in a more structured design.

- **Advantage:** significant reduction in ALM and register usage by delegating storage to dedicated hardware memory resources.
- **Disadvantage:** the introduction of one clock-cycle latency per byte processed, which may reduce the maximum achievable operating frequency due to synchronization constraints.

6.2.3 Motivation for the Comparative Study

The decision to synthesize both versions arises from the need to balance **throughput** and **area utilization**. In complex embedded systems, where the cipher is only one of several FPGA modules, minimizing ALM usage is often a higher priority than maximizing frequency, especially when the latter already exceeds system requirements (100 MHz).

The following sections present the quantitative results that justify the transition from the LUT-based version to the ROM-based version as the final design choice.

6.3 Resource Utilization Analysis

The resource utilization analysis allows evaluation of the physical impact of the design on the Cyclone V architecture. The comparison between the two implementations highlights how the mapping strategy influences the consumption of elementary hardware resources.

The data reported in Table 6.2 show the usage of *Adaptive Logic Modules* (ALMs), dedicated registers, and embedded memory bits.

Table 6.2: Comparison of hardware resources used: LUT vs ROM prototypes

System Resource	LUT Version	ROM Version
Logic utilization (ALMs)	61 / 113 560 (< 1%)	25 / 113 560 (< 1%)
Total registers	19	13
Total block memory bits	0 / 12 492 800 (0%)	2 048 / 12 492 800 (< 1%)
Total pins (I/O)	29 / 378 (8%)	29 / 378 (8%)

6.3.1 Optimization of Combinational Logic

The most significant result is the reduction in combinational logic usage.

The `Stream_cipher_v1_ROM` version uses only **25 ALMs**, representing a **59%** reduction compared to the 61 ALMs required by the LUT-based version.

This improvement is due to the different implementation of the S-Box:

- In the **LUT version**, Quartus synthesizes the substitution table as a large network of logic gates and multiplexers, consuming significant resources to implement a function that is inherently static.
- In the **ROM version**, the table is directly mapped into **M10K** memory blocks. The 8-bit AES S-Box requires 256×8 bits, resulting in a total usage of 2048 memory bits.

6.3.2 Register and I/O Pin Analysis

The register count is also optimized in the ROM version, decreasing from 19 to **13 registers** (a **31%** reduction). This improvement is due to a more streamlined datapath, enabled by the fact that the ROM output is internally registered by dedicated FPGA hardware.

The number of I/O pins remains unchanged at **29** for both implementations, since the external interface defined in the `stream_cipher_top` module does not change structurally.

This efficiency demonstrates that the ROM-based implementation is significantly more suitable for *area-constrained* FPGA designs, where preserving logic resources is critical for integrating additional system modules.

6.4 Static Timing Analysis (STA)

Static Timing Analysis represents a fundamental step in certifying the temporal reliability of the design. Using the **Timing Analyzer** tool, it has been verified that every logic path in the cipher satisfies both *Setup* and *Hold* constraints, preventing the occurrence of metastable states.

6.4.1 Maximum Frequency Results (F_{max})

The maximum operating frequency (F_{max}) is the direct outcome of STA and represents the upper speed limit beyond which the circuit can no longer correctly sample data due to propagation delays. Table 6.3 reports the F_{max} values for both prototypes, evaluated under the most conservative thermal model (*Slow 1100mV 85C*).

Table 6.3: Comparison of Maximum Operating Frequency

Parameter	LUT Version	ROM Version
Fmax (MHz)	107.94 MHz	99.3 MHz
Minimum Period (T_{min})	9.264 ns	10.070 ns

6.4.2 Timing Margin Analysis (Slack) and Timing Closure

To ensure system stability, **Slack** (the safety margin with respect to timing constraints) must remain positive under all operating conditions. A positive *Setup Slack* indicates that the signal reaches the destination register before the next clock edge, while a positive *Hold Slack* ensures that the data remains stable for the required time after sampling.

Timing Constraint Optimization (SDC)

Initially, the *Synopsys Design Constraints* (.sdc) file specified an aggressive target with the parameter:

```
set CLK_PERIOD_NS 7.6
```

corresponding to an operating frequency of approximately 131 MHz. However, timing analysis performed with the *Timing Analyzer* showed that this constraint was too strict for the selected Cyclone V architecture, leading to negative *Slack* (timing violations), particularly in the critical path associated with the ROM memory.

To ensure full system reliability and obtain positive margins across all thermal models, the constraint was recalibrated based on STA results. The parameter was modified to:

```
set CLK_PERIOD_NS 10.3
```

This new value, corresponding to approximately 97 MHz, allowed successful timing closure, ensuring that even in the worst-case scenario (*Slow 0°C* model), the system operates safely.

The final results extracted from Quartus thermal models using the updated constraint are summarized in Table 6.4.

Table 6.4: Setup and Hold Slack Analysis (ns) with 10.3 ns constraint

Thermal Model	LUT Version		ROM Version	
	Setup Slack	Hold Slack	Setup Slack	Hold Slack
Slow 1100mV 85°C	0.236	0.363	0.229	0.377
Slow 1100mV 0°C	0.095	0.336	0.033	0.358
Fast 1100mV 85°C	4.026	0.189	4.217	0.182
Fast 1100mV 0°C	4.169	0.168	4.318	0.172

Discussion of Results:

As shown in the table, adjusting the clock period in the SDC file allowed the design to achieve stable timing behavior. In particular, for the **ROM version** under the *Slow 0°C* model, a safety margin of 0.033 ns was obtained. Although small, this positive value confirms that the hardware operates correctly without sampling errors, thereby validating the choice of the 10.3 ns period as a safe operational limit for the physical implementation.

6.4.3 Interpretation of Results

The data analysis highlights two critical aspects:

1. **Timing Stability:** All Slack values are positive (> 0), confirming that the design is timing-closed at 100 MHz. Even in the worst-case scenario for the ROM version at 0°C (0.033 ns), the safety margin is still respected.
2. **Difference in F_{max} :** The LUT version achieves a higher F_{max} (107.94 MHz) compared to the ROM version (99.3 MHz). This is due to the fact that access to M10K memory introduces a longer critical path, including block memory activation latency, while the LUT implementation relies on a faster combinational network, although less area-efficient.

In conclusion, although the ROM-based version operates very close to the 100 MHz constraint limit, the STA analysis confirms that the system remains robust and reliable across the entire operating temperature and voltage range defined for the Cyclone V device.

6.5 Final Design Justification

After a careful comparative analysis between the combinational logic-based prototype (LUT-based) and the memory-based implementation (ROM-based), the ROM-based version was selected as the final hardware architecture. Although the *Static Timing Analysis* results show a maximum frequency of 99.3 MHz, slightly below the ideal 100 MHz target under the most pessimistic thermal model, several engineering considerations confirm its overall superiority.

6.5.1 Resource Utilization Efficiency (Area Efficiency)

The primary advantage of the proposed architecture lies in the significant reduction of generic logic usage. The ROM-based implementation reduces ALM consumption by **59%** and register usage by **31%** compared to the LUT-based version. From a *System-on-Chip* perspective, preserving ALMs is essential to enable integration of additional functional modules on the same FPGA device.

To quantify this advantage from an engineering perspective, an efficiency metric (E) was defined, relating system *throughput* to silicon area usage:

$$E = \frac{\text{Throughput}}{\text{Area}} \quad \left[\frac{\text{MB/s}}{\text{ALM}} \right] \quad (6.1)$$

Assuming a nominal throughput of 20 MB/s for both architectures (computed at 100 MHz), the comparison yields the following results:

- **LUT Model:**

$$E_{LUT} = \frac{20 \text{ MB/s}}{61 \text{ ALM}} \approx 0.328 \text{ MB/s/ALM}$$

- **ROM Model:**

$$E_{ROM} = \frac{20 \text{ MB/s}}{25 \text{ ALM}} = 0.8 \text{ MB/s/ALM}$$

The results show that the ROM-based implementation is approximately **2.4 times more efficient** than the LUT-based version in terms of logic utilization. This confirms that the use of 2048 bits of M10K memory constitutes a virtually “zero cost” area

solution, as it exploits dedicated hardware resources while freeing general purpose logic for other computational tasks.

In conclusion, the ROM architecture provides significantly higher performance density, making it particularly suitable for *resource constrained* FPGA designs on Cyclone V devices.

6.5.2 Scalability and Routing Predictability

Implementing the S-Box using distributed LUT logic tends to produce a dense and less structured interconnection network, which may lead to routing congestion as system complexity increases. In contrast, the ROM-based implementation:

- provides shorter and more predictable signal paths to dedicated memory blocks;
- reduces the probability of combinational glitches, since the keystream is sampled synchronously;
- ensures greater robustness against variations in Quartus *Place-and-Route* outcomes.

6.5.3 Throughput vs Frequency Analysis

Although the measured maximum frequency is 99.3 MHz (a negligible deviation of 0.7% from the nominal 100 MHz), it is important to note that this value is obtained under the *Worst-Case Scenario* (85°C). Under typical operating conditions, the frequency comfortably exceeds 100 MHz, ensuring the expected throughput of 20 MB/s.

Furthermore, the introduction of the wait state (ST_ROM_WAIT) required by synchronous ROM access does not negatively affect the final throughput, as it has already been fully integrated and validated within the *handshake* protocol (Chapter 4).

Table 6.5: Performance Summary and Hardware Resource Comparison (Intel Cyclone V)

Design	Description	Resources (ALMs)	Max freq. (MHz)	Latency/CPD (cc)	Throughput (Gbps)	Efficiency (Gbps/ALM)
v1_LUT	LUT-based (Combinational)	61	107.94	1	0.16	0.0026
v1_ROM	ROM-based (Synchronous)	25	99.30	5	0.16	0.0064

It is worth noting that the maximum frequency of 99.30 MHz reported in Table 6.5 is strictly bound to the absolute Worst Case Timing Model (Slow 1100mV 0°C), where temperature inversion penalizes the synchronous ROM access times. Under typical operating conditions, the device comfortably operates above 100 MHz, fully satisfying the nominal design constraints without any meta stability risks.

Even though the LUT implementation features a shorter intrinsic combinational path, its throughput is practically capped at 0.16 Gbps by the handshake protocol interfaces. Since the communication protocol forces a steady state rate of 1 byte per 5 clock cycles to ensure safe data sampling, the synchronous architecture (v1_ROM) achieves the exact same operational throughput while drastically reducing logic area.

6.5.4 Implementation Conclusions

In summary, the ROM-based version represents the best trade off between performance and silicon efficiency. The design choice prioritizes architectural simplicity and energy efficiency, making the `Stream_Cipher_V1` core well suited for *security oriented* applications on Cyclone V platforms, where reliability and resource optimization are critical requirements.